Système et Réseaux : TD5

février 2026 — Pierre Rousselin

Exercice 1 : Le problème du coiffeur endormi (Edsger Dijkstra)

Dans un salon de coiffure, il y a N chaises pour que les clients attendent, un coiffeur, et un fauteuil pour se faire couper les cheveux. Les règles sont les suivantes :

- S'il n'y a aucun client, le coiffeur dort dans son fauteuil.
- Un client doit réveiller le coiffeur s'il dort.
- Lorsqu'un client rentre :
  - si les N chaises sont occupées, il repart ;
  - sinon il va s'asseoir sur une des chaises vides et attend que le coiffeur l'appelle.
- Lorsqu'il a fini une coupe, le coiffeur inspecte la salle d'attente pour voir s'il y a encore des clients et s'il n'y en a pas il dort.

```c
#define NUM_CLIENTS 10
#define NUM_CHAISES 6
#define COEFF_ATTENTE 500000
int num_chaises_vides = NUM_CHAISES;
/* Choses à ajouter ici */
void *client(void *arg);
void *coiffeur(void *arg);
void coiffer(void);
void bavarder_avec_le_coiffeur(void);

int main(void)
{
        int i
        pthread_t clt[NUM_CLIENTS], coiff;
        /* Préparatifs : initialisations... */
        /* Choses à faire ici */
        if (pthread_create(&coiff, NULL, coiffeur, NULL) != 0) {
                perror("create coiff");
                return 1;
        }
        for (i = 0; i < NUM_CLIENTS; ++i)
                if (pthread_create(&clt[i], NULL, client, NULL) {
                        perror("create client");
                        return 1;
                }
        pthread_join(coiff, NULL);
        /* le coiffeur ne finit jamais */
        return 0;
}
void *coiffeur(void *arg)
{ /* Fonction à modifier */
        for (;;) { coiffer(); }
}
void *client(void *arg)
{ /* Fonction à modifier */
        for (;;) { bavarder_avec_le_coiffeur(); }
}
void coiffer(void) { /* ... */ }
void bavarder_avec_le_coiffeur(void) { /* ... */ }
```

Modéliser la situation décrite précédemment en modifiant le programme ci-dessus. Indications :

— les clients et le coiffeur sont dans une boucle infinie (les clients vont donc revenir se faire coiffer), pour simplifier ;

— la variable globale `num_chaises_vides` joue un rôle important, toute lecture ou écriture de celle-ci doit être mise dans une section critique ;

— deux choses apparaissent comme une ressource : la disponibilité d'un client et la disponibilité du coiffeur.

--- * ---

Exercice 2 : L'oracle et le singe, d'après le partiel 1 2021 – 2022

La situation est la suivante :

— un problème (peut-être très difficile), portant sur les entiers est posé ;

— un oracle sait, lorsqu'on lui propose un entier, vérifier s'il est solution du problème, et c'est tout ce qu'il sait faire ;

— un singe sait proposer des entiers (sans doute au hasard, mais peu importe) et c'est tout ce qu'il sait faire.

Le programme suivant illustre cette situation, l'oracle et le singe sont deux *threads*. Certains détails ont été omis.

```c
void *oracle(void *arg);
void *singe(void *arg);
/* retourne une proposition (un entier positif) */
int inventer_proposition(void);
/* retourne 1 si la proposition prop est solution du problème et 0 sinon */
int verifier_proposition(int prop);

/* Au départ proposition contient -1 qui n'est jamais solution */
int proposition = -1, trouve = 0;

int main(void)
{
        pthread_t s, o;
        pthread_create(&o, NULL, oracle, NULL);
        pthread_create(&s, NULL, singe, NULL);
        return 0;
}

void *singe(void *arg)
{
        while (!trouve)
                proposition = inventer_proposition();
        return NULL;
}
void *oracle(void *arg)
{
        while (!trouve)
                trouve = verifier_proposition(proposition);
        printf("Le singe a trouvé la solution ! C'est %d !\n", proposition);
        return NULL;
}
int inventer_proposition(void) { /* ... */ }
int verifier_proposition(int prop) { /* ... */ }
```

Sauf mention contraire, le contenu et la durée d'exécution de `inventer_proposition` et `verifier_proposition` sont quelconques.

1. On suppose que les deux *threads* s'exécutent sur deux processeurs différents, qui sont en permanence disponibles. De plus, on suppose que le singe met toujours 2 unités de temps pour inventer une proposition tandis que l'oracle en met 7 pour vérifier qu'une proposition est correcte. Tous les autres temps sont négligés.

   On suppose enfin que le singe adopte ici la stratégie d'essayer tous les entiers les uns après les autres : il invente donc 0, 1, 2, ... Que se passe-t-il si la solution est 3 ? (quelles événements à quelle date, contenu de `proposition`).

2. **Première solution.** Faire en sorte (décrire les changements dans le programme) en utilisant sémaphores et/ou mutex et/ou variables supplémentaires, que :
   — l'oracle attend la proposition du singe avant de vérifier celle-ci ;
   — le singe attend alors la vérification de l'oracle avant d'inventer une nouvelle proposition.

   Quel est l'inconvénient de cette méthode ?

3. **Deuxième solution.** Faire en sorte que le singe soit en train d'inventer *la prochaine proposition* pendant que l'oracle examine la proposition courante.

   Décrire les modifications à apporter par rapport au programme écrit ci-dessus (utiliser mutex et/ou sémaphores et/ou variables supplémentaires).

4. **Troisième solution.** Dans cette solution, une file est utilisée : lorsqu'il a inventé une proposition, le singe la met au bout de la file et l'oracle vérifie chaque fois la proposition en tête de file.

   Les files sont implémentées (peu importe comment) de manière à ne pas être bornées et vous avez accès aux fonctions suivantes :

```
/* retourne un pointeur vers une nouvelle file vide */
file *file_vide(void);
/* enfile l'élément e au bout de la file d'adresse f */
void enfiler(file *f, int e);
/* défile l'élément en tête de file et retourne la valeur ainsi défilée
 * précondition : f est non vide */
int defiler(file *f);
/* libère la mémoire associée à la file d'adresse f */
void file_detruire(file *f);
```

   Décrire les modifications par rapport au programme écrit ci-dessus (utiliser mutex et/ou sémaphores et/ou variables supplémentaires).

--- * ---

Exercice 3 : Lecteurs et écrivains, encore

On rappelle la solution classique du problème des lecteurs et des écrivains (les parties les moins intéressantes ont été omises).

```
int num_lect_bib = 0;
sem_t bib_est_vide;
pthread_mutex_t update_num_lects_bib;
void *lecteur(void *arg);
void *ecrivain(void *arg);
int main(void)
{
        pthread_mutex_init(&update_num_lects_bib, NULL);
        sem_init(&bib_est_vide, 0, 1);
        /* création de lecteurs et écrivains... */
        return 0;
}
void *ecrivain(void *arg)
{
        sem_wait(&bib_est_vide);
```

```
16          ecrire();
17          sem_post(&bib_est_vide);
18  }
19  void *lecteur(void *arg)
20  {
21          pthread_mutex_lock(&update_num_lects_bib);
22          if (num_lect_bib == 0)
23                  sem_wait(&bib_est_vide);
24          ++num_lect_bib;
25          pthread_mutex_unlock(&update_num_lects_bib);
26          lire();
27          pthread_mutex_lock(&update_num_lects_bib);
28          --num_lect_bib;
29          if (num_lect_bib == 0)
30                  sem_post(&bib_est_vide);
31          pthread_mutex_unlock(&update_num_lects_bib);
32  }
```

1. On suppose que `lire` prend 2 unités de temps, qu'écrire en prend 5, qu'il y a autant de processeurs disponibles que l'on veut et que tout ce qui n'est pas lire et écrire a une durée négligeable.

a) Que se passe-t-il si trois lecteurs arrivent au temps 0, 1 et 2 et qu'un écrivain arrive au temps 1 ?

b) Que se passe-t-il si un écrivain arrive au temps 0 et deux lecteurs au temps 1 ?

2. Construire (toujours avec les hypothèses précédentes) une situation de famine pour l'écrivain avec 3 *threads* dont 2 lecteurs en boucles infinies et 1 écrivain.

3. On dit que la solution précédente donne priorité aux lecteurs. La modifier, de telle sorte que, lorsqu'un écrivain arrive à la bibliothèque, les lecteurs qui y sont déjà finissent leurs lectures mais aucun nouveau lecteur ne peut entrer dans la bibliothèque avant l'écrivain.

--- * ---